Name: Ranjitha.R

Reg.No:192421285

Github link:

Ex 11 Create a Static Website and Containerize, Build & Serve it using Docker. Tasks:

1. Create a Simple Static Website (index.html file) with basic HTML content.

2. Write/create a Dockerfile to serve the website using Nginx.

3. Build the Docker Image

4. Run the container:

5. Access the Website using a Browser

AIM

To create a basic static website, package it into a Docker image, and serve it through an Nginx container.

PROCEDURE

1. Create a project folder and add an **index.html** file containing the website heading and message.
2. Create a **Dockerfile** using the Nginx image as the base image and copy the website file into Nginx's web root.
3. Build the Docker image using the **docker build** command.
4. Start a container from the generated image and map a host port to the container's Nginx port.
5. Open the mapped localhost address in a web browser and verify that the page loads.

INPUT

index.html containing the website content, the Dockerfile, an image name, and a host-to-container port mapping.

OUTPUT

A Docker image is created and an Nginx container serves the static website through the browser.



RESULT

Thus, Experiment 11 was carried out according to the given tasks. The expected output and the screenshot evidence are recorded in the original lab document

Ex 12 Create a Simple Python Flask API, Containerize the Application, Build & Push the Image using Docker and Deploy the Application using Kubernetes.

Tasks:

1. **Create a Simple Flask API by writing a Python file (app.py) with basic endpoints.**
2. **Containerize the Flask App using Dockerfile.**
3. **Build the Image using Docker.**
4. **Push the Image to Docker Hub.**
5. **Create Kubernetes manifests (Deployment YAML & Service YAML) to deploy the application.**

AIM

To develop a simple Python Flask API, package it with Docker, push the image to Docker Hub, and deploy it using Kubernetes.

PROCEDURE

1. Create **app.py** with Flask routes, including a home route and a health-check route.
2. Create a requirements file and a Dockerfile to install dependencies and run the Flask application.
3. Build the Docker image, log in to Docker Hub, tag the image with the repository name, and push it.
4. Write Kubernetes Deployment and Service YAML files using the pushed image.
5. Apply the manifests with **kubectl apply** and check the deployment, pods, and service using kubectl commands.
6. Access the service and verify that the API responds correctly.

INPUT

Flask source file, requirements file, Dockerfile, Docker Hub repository/image tag, and Kubernetes Deployment and Service YAML files.

OUTPUT

The Flask API is packaged and pushed to Docker Hub, then runs as a Kubernetes pod and is exposed through a service.

1. Flask API — app.py

```
# app.py
$ from flask import Flask, jsonify
$ app = Flask(__name__)
$
$ @app.route("/")
$ def home():
    return jsonify(message="Hello from Flask API!")
$
$ @app.route("/health")
$ def health():
    return jsonify(status="OK")
$
$ if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

sri.balaji@example.com

2. Dockerfile — Containerize Flask App

```
# Dockerfile
$ FROM python:3.11-slim
$ WORKDIR /app
$ COPY requirements.txt .
$ RUN pip install --no-cache-dir -r requirements.txt
$ COPY app.py .
$ EXPOSE 5000
$ CMD ["python", "app.py"]
$
# requirements.txt
$ flask==3.0.3
```

sri.balaji@example.com

3. Build & Push Docker Image

```
$ docker build -t sribalaji/flask-api:1.0 .
$ Successfully built abc123def456
$ Successfully tagged sribalaji/flask-api:1.0
$
$ docker login
$ Login Succeeded
$
$ docker push sribalaji/flask-api:1.0
$ 1.0: digest: sha256:9f2c... size: 1571
$ The push refers to repository [docker.io/sribalaji/flask-api]
```

sri.balaji@example.com

4. Kubernetes Deployment & Service

```
# kubectl apply -f deployment.yaml
$ deployment.apps/flask-api-deployment created
# kubectl apply -f service.yaml
$ service/flask-api-service created
$
# kubectl get pods
$ NAME                          READY  STATUS    RESTARTS  AGE
$ flask-api-7d9f8b6c5-x2k9p     1/1    Running   0         12s
$
# kubectl get svc
$ NAME              TYPE       CLUSTER-IP     PORT(S)
$ flask-api-service  NodePort   10.96.45.12    5000:30080/TCP
$ Application deployed successfully
```

sri.balaji@example.com

RESULT

Thus, Experiment 12 was carried out according to the given tasks. The expected output and the screenshot evidence are recorded in the original lab document.

Ex 13 Set up a CI/CD pipeline to automate the building, testing, and deployment of a containerized application.

Tasks:

1. Set up Jenkins on a local machine or server.

2. Create a Dockerfile to containerize a sample application.

3. Write a Jenkinsfile to automate the process of building the Docker container, running tests, and deploying to a cloud platform (e.g., AWS or GCP).

4. Configure Jenkins to trigg`er builds upon code commits or pull requests.

AIM

To automate the building, testing, and deployment of a containerized application using Jenkins.

PROCEDURE

1. Install and start Jenkins on a local machine or server and configure the required tools and credentials.
2. Create a sample application and write a Dockerfile to containerize it.
3. Add a **Jenkinsfile** to the repository with stages for checkout, build, test, push, and deployment.
4. Configure a Jenkins pipeline job to use the repository and Jenkinsfile.
5. Enable build triggers for code commits or pull requests.
6. Run the pipeline and inspect each stage's console output to confirm whether the build, tests, and deployment complete.

INPUT

Application source code, Dockerfile, Jenkinsfile, repository URL, and any required registry/cloud credentials.

OUTPUT

Jenkins executes the defined pipeline stages automatically and reports the build/test/deployment status.

1. Jenkins Setup on Local Machine

```
# Install Jenkins with Docker
$ docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  jenkins/jenkins:lts
$
# Get initial admin password
$ docker exec jenkins cat /var/jenkins_admin_password
$
$ Jenkins is fully up and running
$ Jenkins URL: http://localhost:8080
$ Plugins installed: Git, Docker, Pipeline, GitHub Integration
```

2. Dockerfile for Sample Application

```
# Dockerfile
$ FROM node:18-alpine
$ WORKDIR /app
$ COPY package*.json ./
$ RUN npm install --production
$ COPY . .
$ EXPOSE 3000
$ CMD ["npm", "start"]
$
# Test locally
$ docker build -t sri-balaji-sample-app:latest .
$ docker run -d -p 3000:3000 sri-balaji-sample-app:latest
```

3. Jenkinsfile — Build, Test, Deploy

```
pipeline {
  agent any
  environment { DOCKER_IMAGE = "sribalaji/sample-app" }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Build') { steps { sh 'docker build -t DOCKER_IMAGE:BUILD_NUMBER .' } }
    stage('Test') { steps { sh 'npm test' } }
    stage('Push') { steps { sh 'docker push DOCKER_IMAGE:BUILD_NUMBER' } }
    stage('Deploy') { steps { sh 'kubectl set image deployment/sample-app app=DOCKER_IMAGE:BUILD_NUMBER' } }
  }
  post { success { echo 'Deployment successful' } }
}
```

4. Jenkins Pipeline Build Output

```
$ Started by GitHub push by sri-balaji
$ Checking out Revision 3f9c2ab (main)
$
$ [Pipeline] stage
$ [Pipeline] { Build }
$ Successfully built 8f2a91c4d7e8
$ Successfully tagged sribalaji/sample-app:17
$
$ [Pipeline] stage
$ [Pipeline] { Test }
$ Test Suites: 4 passed, Tests: 12 passed
$
$ [Pipeline] stage
$ [Pipeline] { Deploy }
$ deployment.apps/sample-app image updated
$
$ Finished: SUCCESS
$ Build triggered automatically on code commit / pull request
```

sri.balaji@example.com | Illustrative screenshot – not actual execution

RESULT

Thus, Experiment 13 was carried out according to the given tasks. The expected output and the screenshot evidence are recorded in the original lab document.

Ex 14 Implement Continuous Deployment using GitHub Actions to deploy Dockerized application.

Tasks:

1. Set up a GitHub repository and push a simple Dockerized application.

2. Create a GitHub Actions workflow to automatically build and push the Docker image to Docker Hub or GitHub Container Registry.

3. Automate deployment to a cloud platform (e.g., AWS ECS, Azure Kubernetes Service, or Google Kubernetes Engine).

4. Test the CI/CD pipeline by pushing new code changes and verifying that the deployment occurs automatically.

AIM

To automate Docker image building, publishing, and application deployment using a GitHub Actions workflow.

PROCEDURE

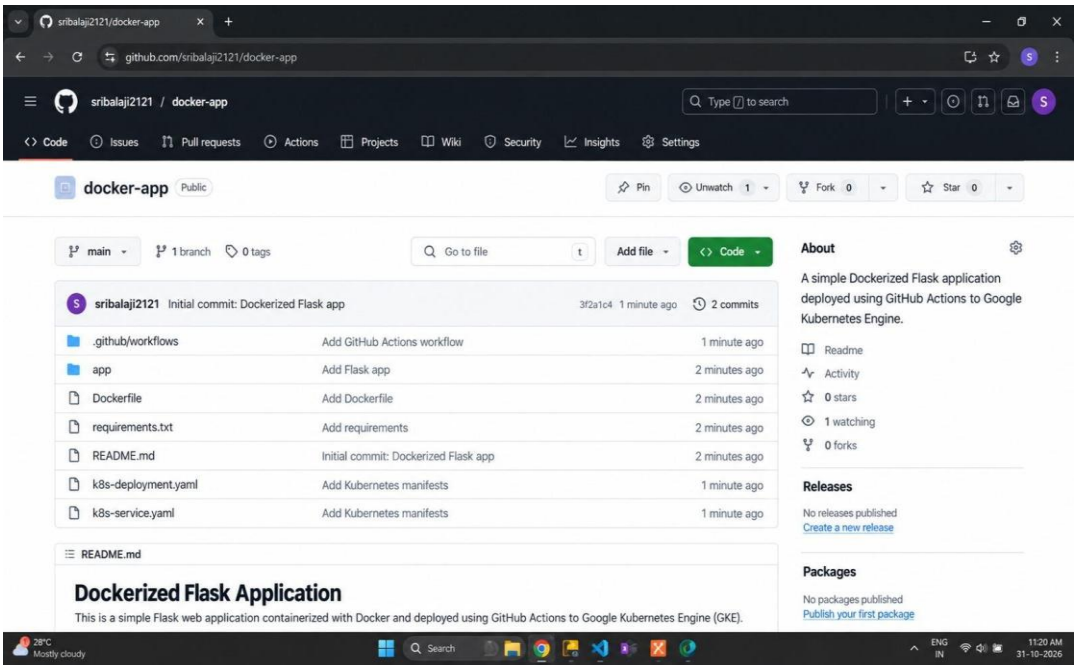
1. Create a GitHub repository and push the Dockerized application source code and Dockerfile.
2. Create a workflow YAML file inside **.github/workflows**.
3. Configure workflow triggers, such as pushes to the main branch.
4. Add steps to check out the repository, build the Docker image, authenticate to the container registry using repository secrets, and push the image.
5. Add the deployment step for the selected cloud platform and configure the required credentials securely.
6. Push a code change and open the Actions tab to verify that the workflow runs and deployment completes.

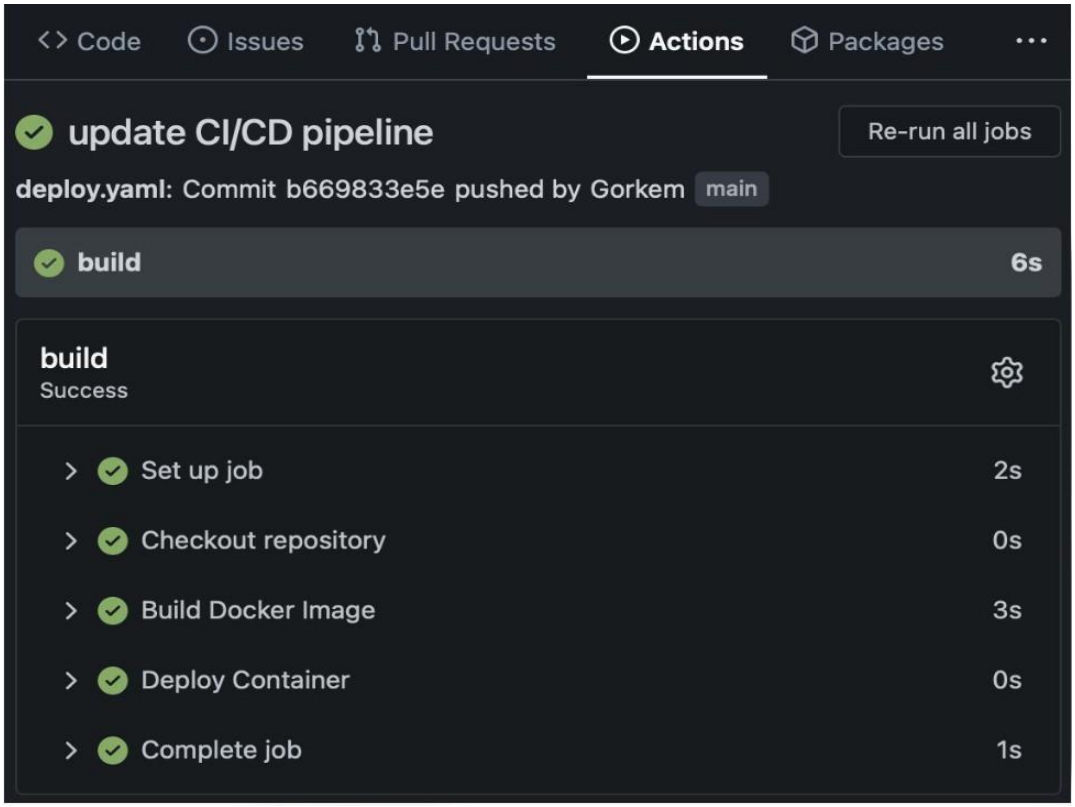
INPUT

GitHub repository, application source code, Dockerfile, workflow YAML file, registry details, and deployment secrets.

OUTPUT

A code push triggers the GitHub Actions workflow, which builds and publishes the Docker image and performs the configured deployment.


sribalaji2121/docker-app
github.com/sribalaji2121/docker-app
sribalaji2121 / docker-app
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
docker-app
Public
Pin
Unwatch 1
Fork 0
Star 0
main
1 branch
0 tags
Go to file
t
Add file
Code
sribalaji2121 Initial commit: Dockerized Flask app
3f2a1c4 1 minute ago
2 commits
.github/workflows Add GitHub Actions workflow 1 minute ago
app Add Flask app 2 minutes ago
Dockerfile Add Dockerfile 2 minutes ago
requirements.txt Add requirements 2 minutes ago
README.md Initial commit: Dockerized Flask app 2 minutes ago
k8s-deployment.yaml Add Kubernetes manifests 1 minute ago
k8s-service.yaml Add Kubernetes manifests 1 minute ago
README.md
Dockerized Flask Application
This is a simple Flask web application containerized with Docker and deployed using GitHub Actions to Google Kubernetes Engine (GKE).
About
A simple Dockerized Flask application deployed using GitHub Actions to Google Kubernetes Engine.
Readme
Activity
0 stars
1 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
28°C
Mostly cloudy
Search
ENG
IN
11:20 AM
31-10-2026



RESULT

Thus, Experiment 14 was carried out according to the given tasks. The expected output and the screenshot evidence are recorded in the original lab document.

Ex 15 Create a GitHub Repository and Implement Version Control Tasks:

- **Set up a repository for a team project.**
- **Create branches for individual modules.**
- **Merge branches with pull requests after code reviews.**
- **Resolve conflicts during branch merging.**
- **Document the workflow using the README file.**
- **Submit the repository li**

AIM

To manage a team project using Git and GitHub through repositories, branches, pull requests, code reviews, and conflict resolution.

PROCEDURE

1. Create a GitHub repository for the team project and add a README file describing the project.

2. Clone the repository to the local system using **git clone**.

3. Create separate branches for individual modules using **git branch** or **git switch -c**.

4. Make changes in each branch, stage them, and commit them with meaningful messages.

5. Push the branches to GitHub and create pull requests for code review.

6. After review, merge the pull requests into the main branch.

7. If merge conflicts occur, edit the affected files, remove conflict markers, test the changes, and commit the resolution.

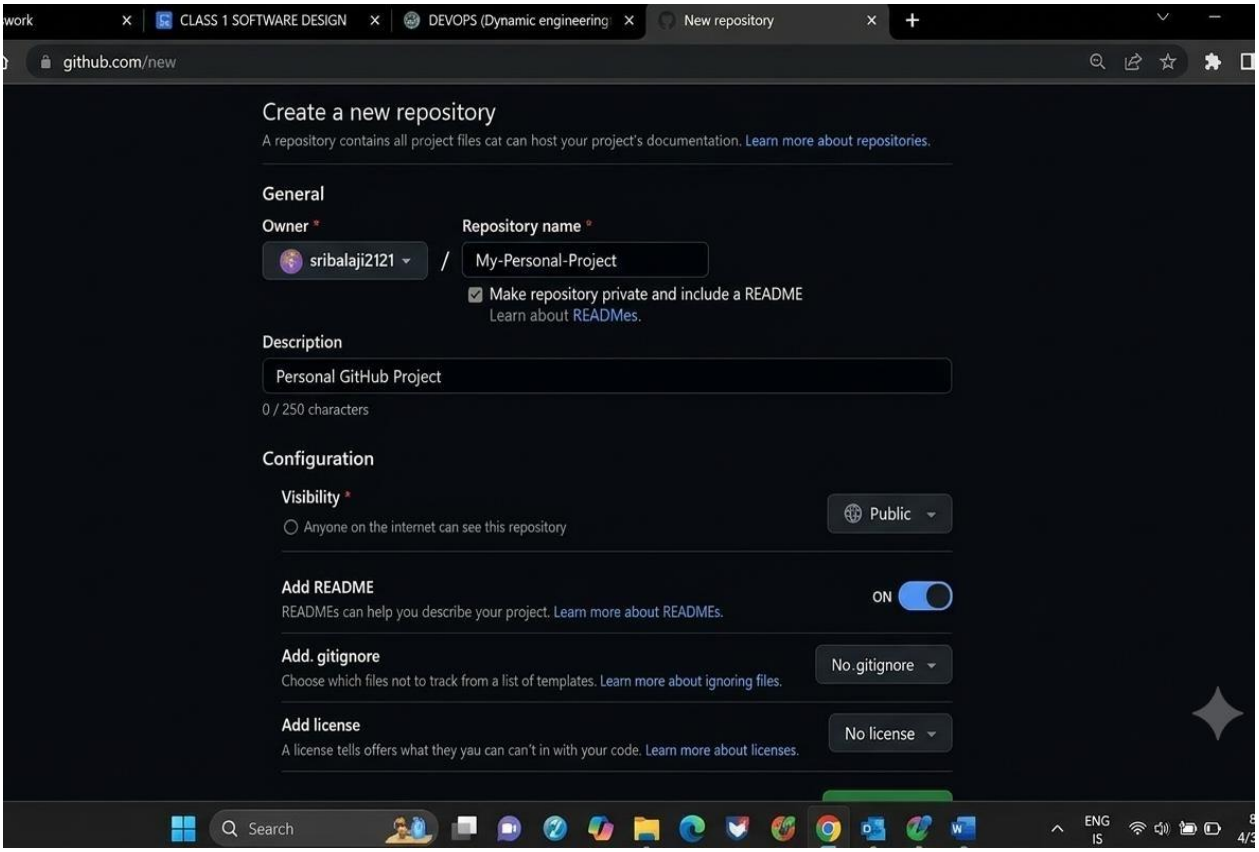
8. Update the README with the project details and version-control workflow.

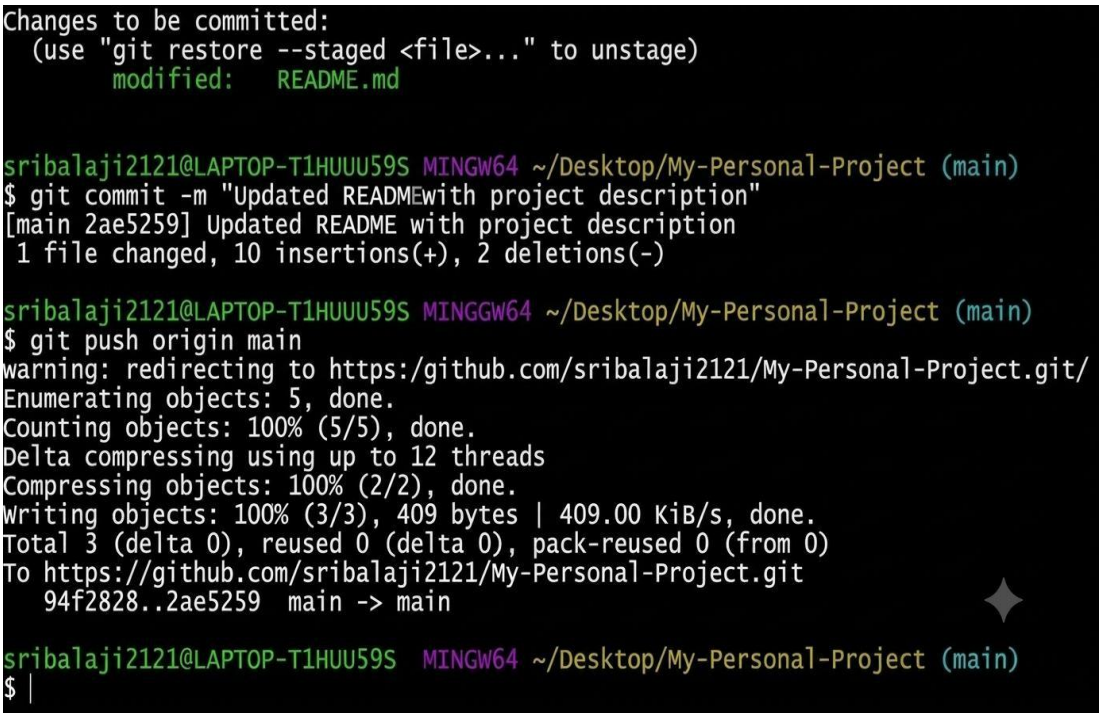
INPUT

GitHub repository URL, project files, branch names, commit messages, and README documentation.

OUTPUT

A shared GitHub repository is maintained with module branches, reviewed pull requests, merged changes, and documented project workflow.


work
CLASS 1 SOFTWARE DESIGN
DEVOPS (Dynamic engineering
New repository
github.com/new
Create a new repository
A repository contains all project files cat can host your project's documentation. Learn more about repositories.
General
Owner *
Repository name *
sribalaji2121
/
My-Personal-Project
Make repository private and include a README
Learn about READMEs.
Description
Personal GitHub Project
0 / 250 characters
Configuration
Visibility *
Anyone on the internet can see this repository
Public
Add README
READMEs can help you describe your project. Learn more about READMEs.
ON
Add. gitignore
Choose which files not to track from a list of templates. Learn more about ignoring files.
No.gitignore
Add license
A license tells offers what they yau can can't in with your code. Learn more about licenses.
No license
Search
ENG
IS



RESULT

Thus, Experiment 15 was carried out according to the given tasks. The expected output and the screenshot evidence are recorded in the original file